

Lecture 2 – Requirement Engineering

CPE334 – Software Engineering (in the Age of AI Coding Agents)

Week#2: 10-14 August 2026

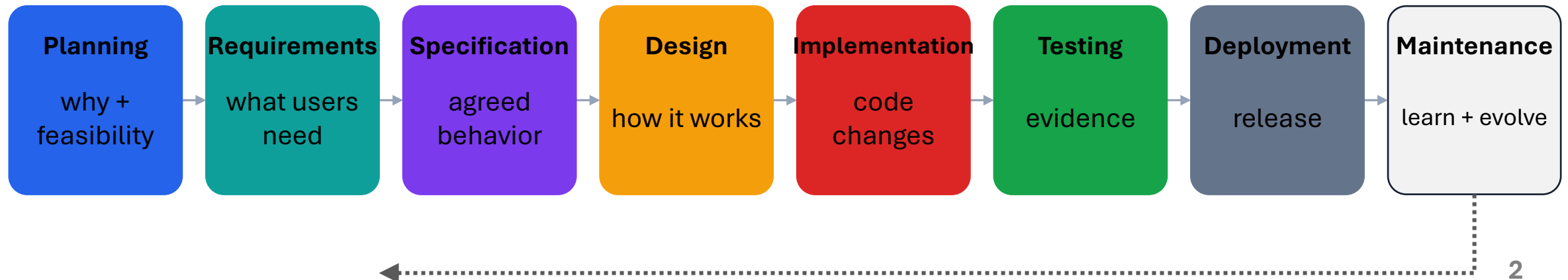
Dr. Suthep Madarasmi
Dr. Piyanit Ua-areemitr
Dr. Santawat Thanyadit

Department of Computer Engineering
King Mongkut's University of Technology Thonburi

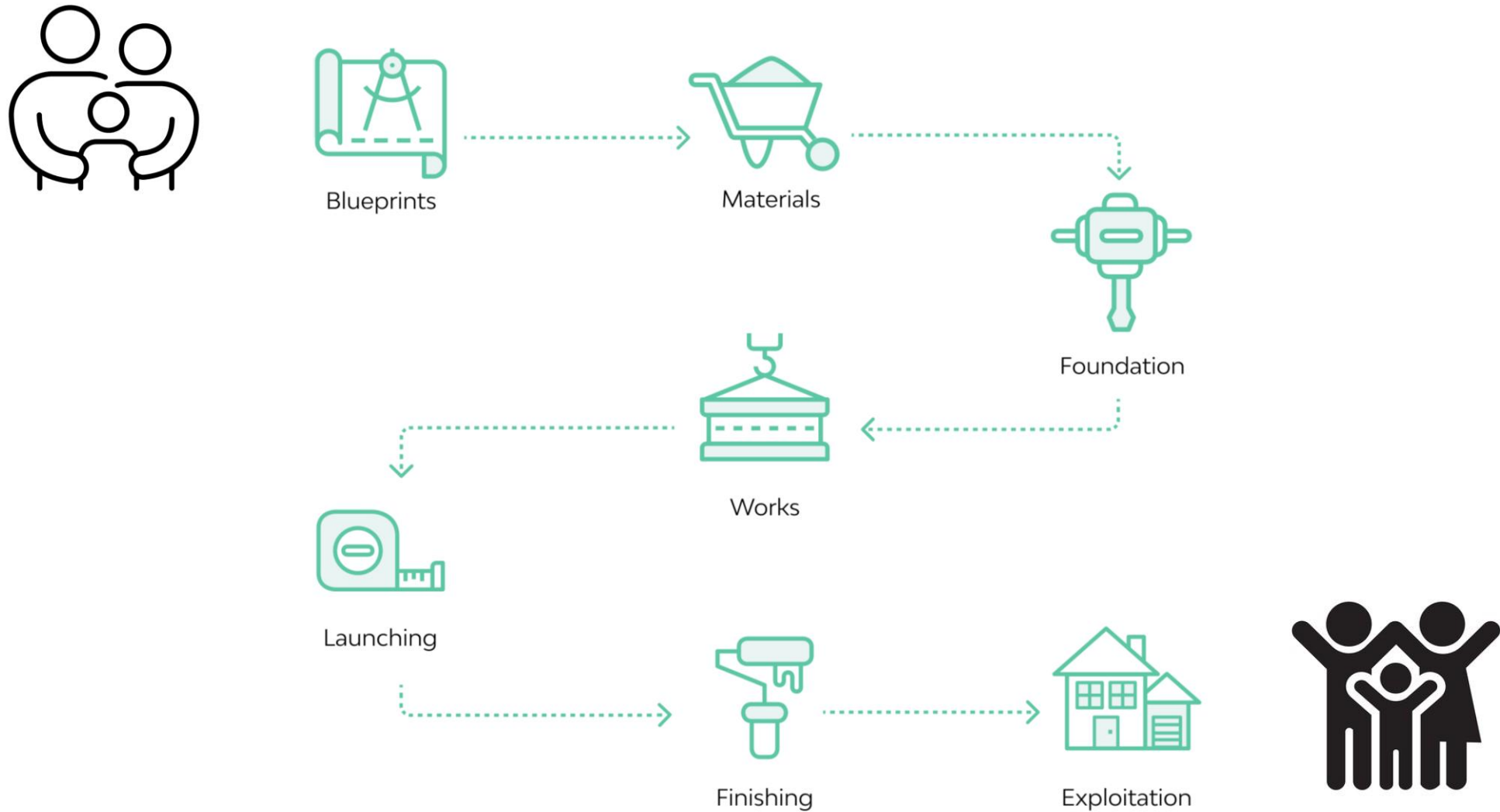


Recap

- Writing code vs engineering a software product
- SDLC
 - Planning style (predictive/adaptive) & delivery styles (incremental/iterative)
 - Models: waterfall, incremental, scrum, kanban
 - Definition of Done
- Version Control
 - Git: workflows (repo, code review, branch)
 - Github



Software can't be touch.



Source: <https://djangostars.com/blog/software-development-process-as-house-building/>

Software complexity

- Application domain: complex problem, domain-expert developer
- Communication among stakeholders
 - Different background knowledge
 - Ambiguous human language
- Dealing with complexity:
 - Design **goals** with **constraints** (time/cost/conflict)

correct	efficient	evolvable	interoperable	maintainable
portable	productive	reliable	repairable	reusable
robust	timely	usable	verifiable	visible

Clear design goals → less complexity of system design

- Divide & conquer: interaction between modules

There is a limit to human understanding

Software engineering activities

- Modeling activities
 - Requirement model – user requirements
 - Solution model – system to be built
- 
- Problem solving activities
 - Appropriate solution in the presence of change
 - Not algorithmic but systematic
 - Knowledge acquisition activities
 - Non-linear process and may need to unlearn
 - Rationale management activities
 - Assumption and solution changes
 - Revisit previous decisions & answer “Why did we make this choice?”

Today's lecture storyline

From vague request to scoped, testable, traceable work.

- 1 **Requirement Engineering**
- 2 **Think like a proposal**
- 3 **Trace evidence**
- 4 **Lab preparation**

A stakeholder request is not yet a specification

Learn to slow down before coding.

**“We need a simple event ticketing app for campus activities.
Students should browse categories and know the system is working. It should be easy to use and reliable.”**

? **Should we just ask AI agent to build the app?**

What is stated?

campus activities · browse categories ·
system health · easy and reliable

What is missing?

user roles · category source · failure
behavior · acceptance standard

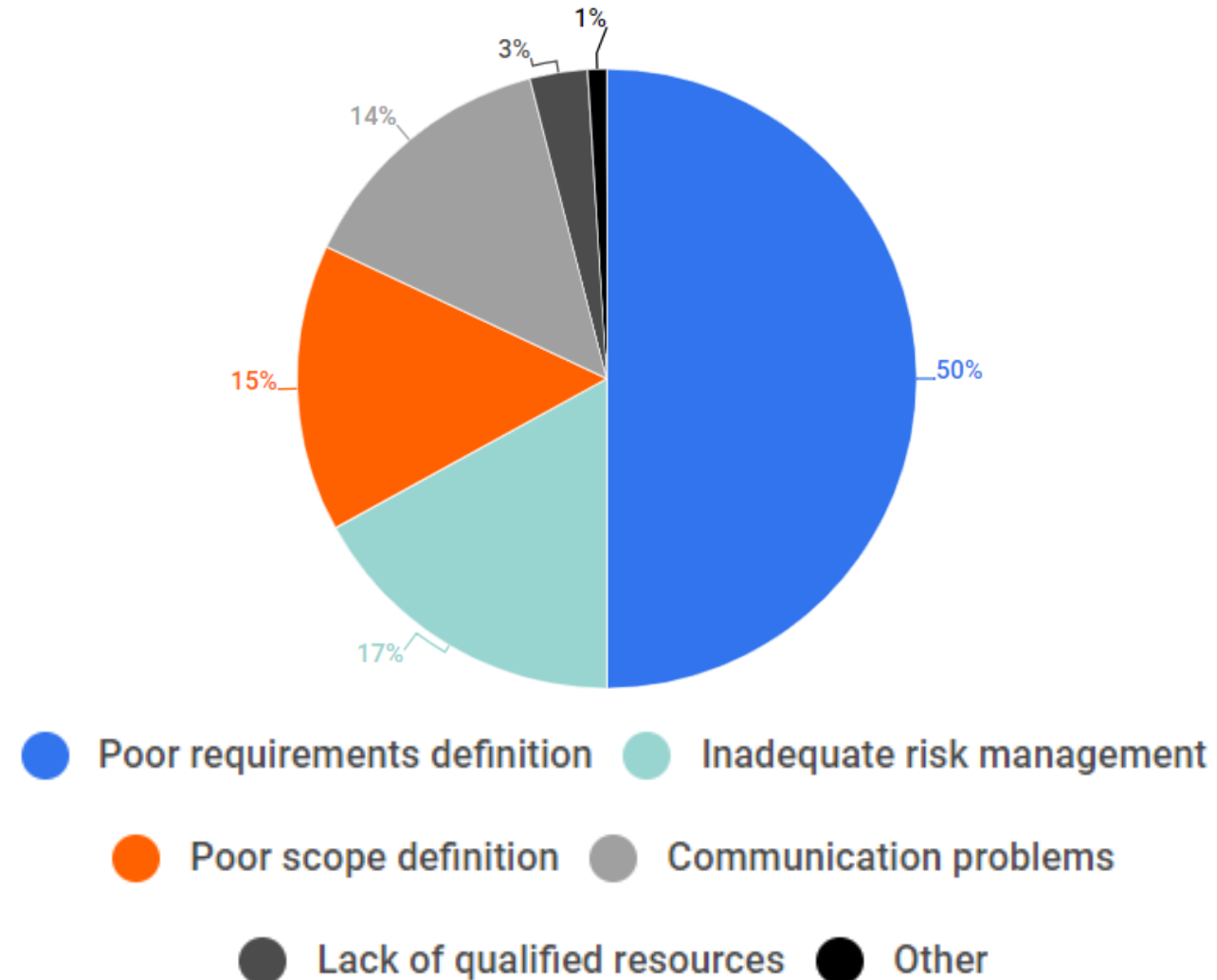
What must be decided?

scope · priority · risks · evidence for
“done”

Challenges from bad requirements

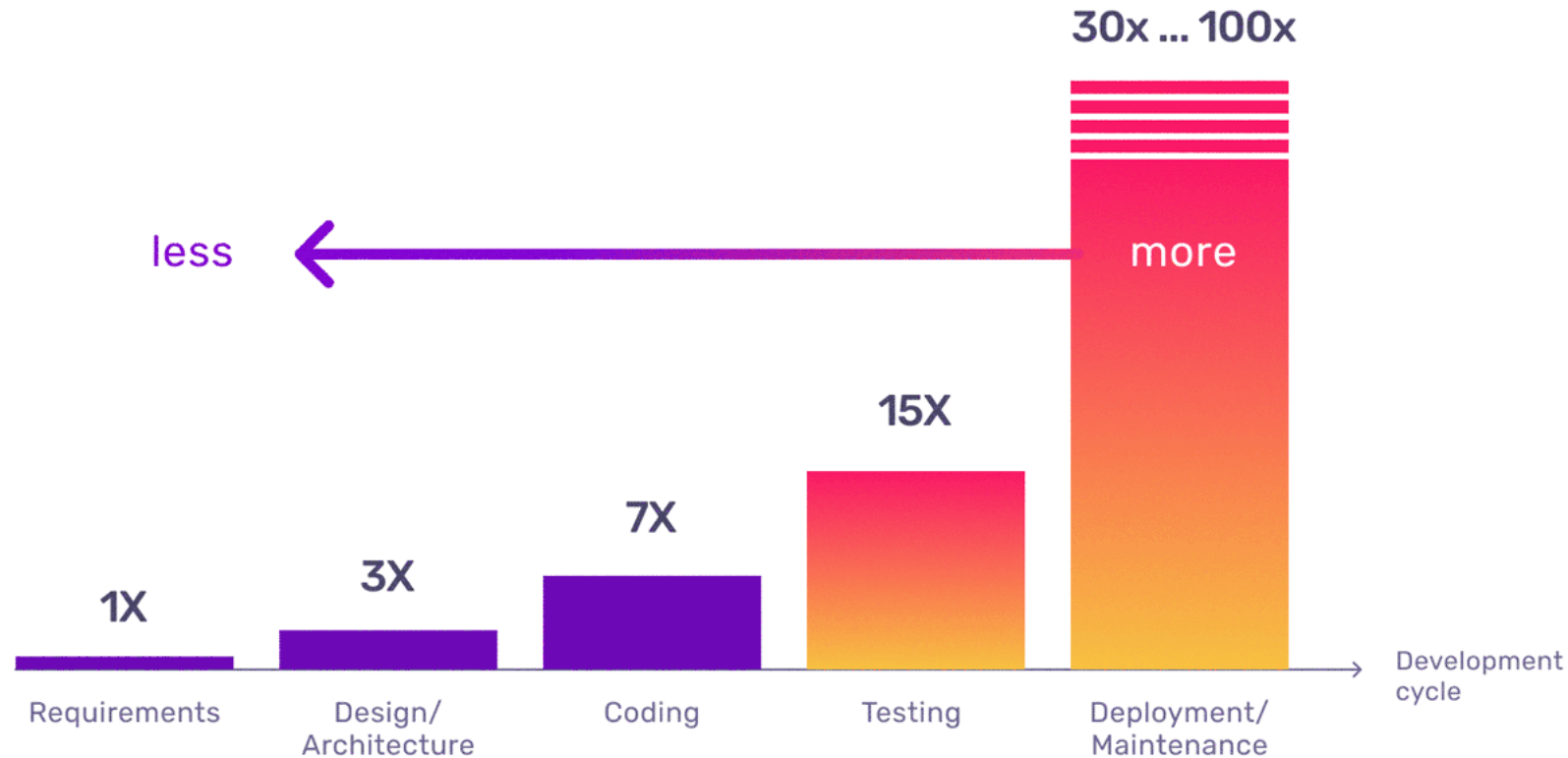
- Ambiguous requirements → guessing
 - “user-friendly”
 - “fast”
- Stakeholder involvement
 - Customer:
“Just one more feature... that can’t be hard!”
 - Requirement writer:
“This isn’t clear, so we’ll assume ...”
 - Developer:
“I bet they’d like this too!”

TOP REASONS WHY PROJECTS FAIL



Source: https://www.zippia.com/advice/project-management-statistics/#project_manager_statistics

Cost of Defects



The more time we save your team, the more time they have to find bugs sooner.

That Saves Money

Requirements Engineering

Make the target clear before generating code.

Goal

Read a stakeholder request and produce **shared understanding**.

Core skills

Clarify assumptions, identify stakeholders, define scope, and separate requirement types.

Output

A small work item with **acceptance criteria** and evidence expectations.

Identify stakeholders before identifying features

Different people care about different outcomes and risks.

Stakeholder - any person or organization who is affected by the system in some way and so who has a legitimate interest

- Relevant position in organization
- Decision-maker for the system-to-be
- Level of domain expertise
- Influence in system acceptance

End users

People who use the product and can describe the tasks needed to perform.

Customer

Individual or organization who derives direct or indirect benefit from the product.

Developers

People who implement, test, and maintain the code.

Affected parties

Anyone affected by either positive or negative project outcomes.

Requirement

Requirements specifies the **problem** not the solution.

- Specific descriptions of your client's needs

- Features that the system must have
- Constraints that the system must satisfy



- A high-level abstract statement, graphical models, design description language, and mathematical specification
- Classification of software requirements
 - **User requirements** – for clients
Services the system provides and its operational constraints
Originate from authorities & add value to someone
 - **System requirements** – for both clients & developers
A structured document defining what should be implemented (functions, services, constraints) **Possible to implement**
 - **Business requirements** – for people with the money
Business goals and objectives expected to achieve e.g. revenue, user satisfaction, etc.

Example: user requirements and system requirements

User requirements definition

- 1.** The Mentcare system shall generate monthly management reports showing the cost of drugs prescribed by each clinic during that month.

User
requirements

Client managers
System end-users
Client engineers
Contractor managers
System architects

System requirements specification

- 1.1** On the last working day of each month, a summary of the drugs prescribed, their cost and the prescribing clinics shall be generated.
- 1.2** The system shall generate the report for printing after 17.30 on the last working day of the month.
- 1.3** A report shall be created for each clinic and shall list the individual drug names, the total number of prescriptions, the number of doses prescribed and the total cost of the prescribed drugs.
- 1.4** If drugs are available in different dose units (e.g. 10mg, 20mg, etc) separate reports shall be created for each dose unit.
- 1.5** Access to drug cost reports shall be restricted to authorized users as listed on a management access control list.

System
requirements

System end-users
Client engineers
System architects
Software developers

Type of requirements

A useful specification says both what the system does and how well it must do it.

Functional requirements – what the system should (not) do

Observable behaviors the system must provide.

Example: The software should be able to show all categories recorded in the database.

Non-functional requirements – what the system should perform

Quality attributes and operating expectations.

Example: The system must have more than 90% up time.

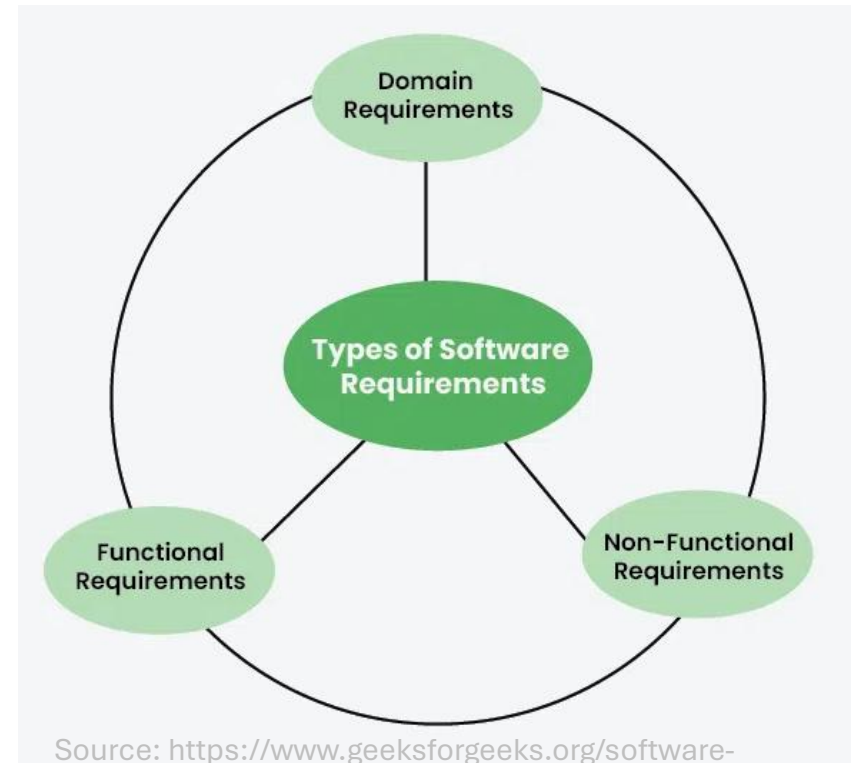
Domain requirements

Rules from the domain or organization.

Example: Category names must be unique.

Acceptance criteria

Feature-specific checks used to accept or reject the work.



Source: <https://www.geeksforgeeks.org/software-engineering/software-engineering-classification-of-software-requirements/>

Functional requirements describe behavior

They should be observable from the outside.

- Describe the system services in detail.
- Ambiguous requirements may be interpreted in different ways by developers and users.
“Search for a student name”

Good FR style

The system shall + action + object + condition.

Avoid vague words

simple · easy · reliable · user-friendly without a check.

Make it testable

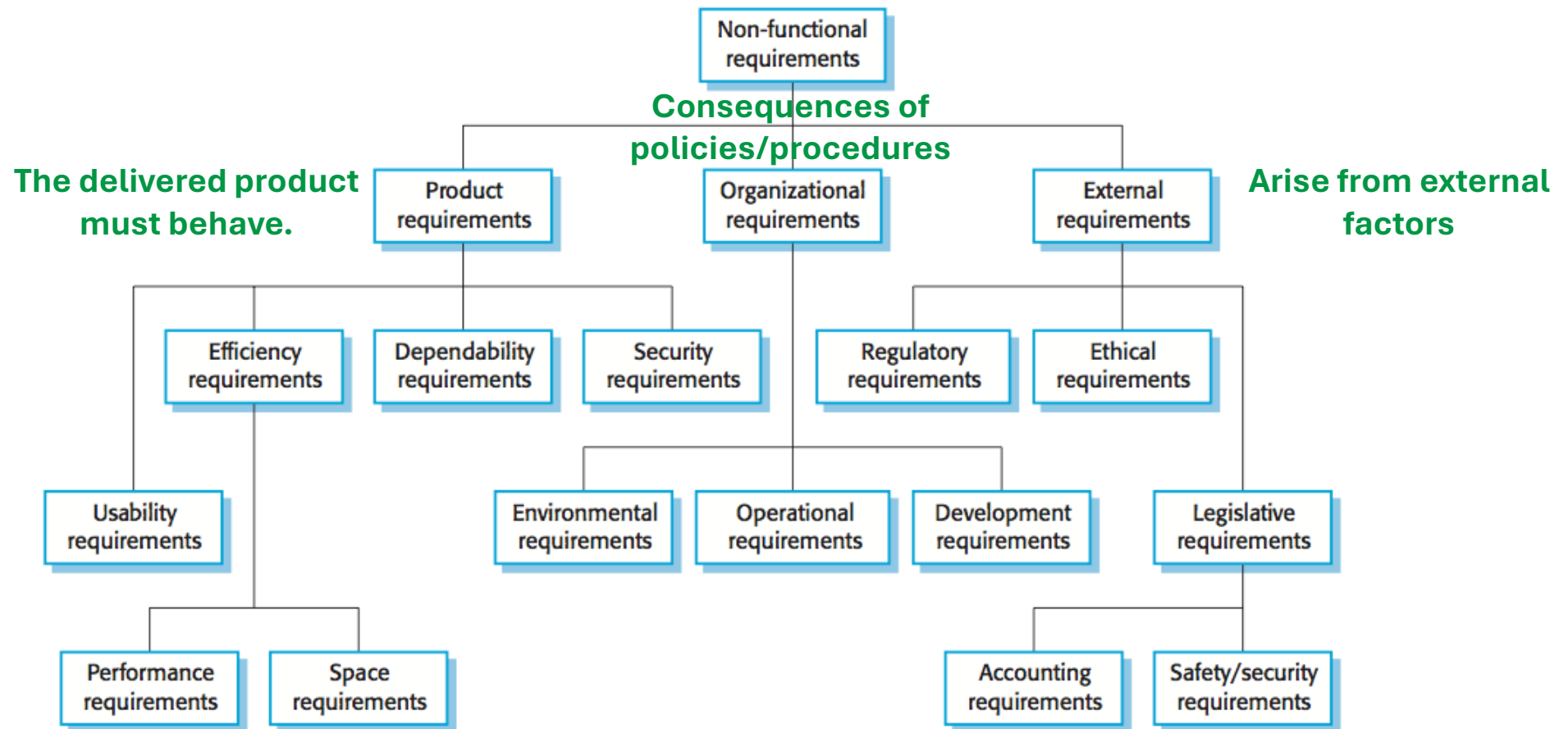
A reviewer should know what output to inspect.

FR-CAT-01: The software should be able to show all categories recorded in the database.

FR-AUT-01: The user must be able to log in to the system with a correct username and a correct password.

Non-functional requirements describe quality

- May be more critical than functional requirements.
- Metrics for specifying NFR
For example, **ease of use** can be measured by training time or number of help frames.



Requirement quality checklist

Students can use this before creating an Issue or asking an AI agent to implement.

Clear

One meaning; no vague adjectives without examples.

Testable

A reviewer can inspect behavior or evidence.

Feasible

Possible within tools, time, and skill level.

Relevant

Connected to a stakeholder need or learning goal.

Atomic

Small enough to implement and review separately.

Traceable

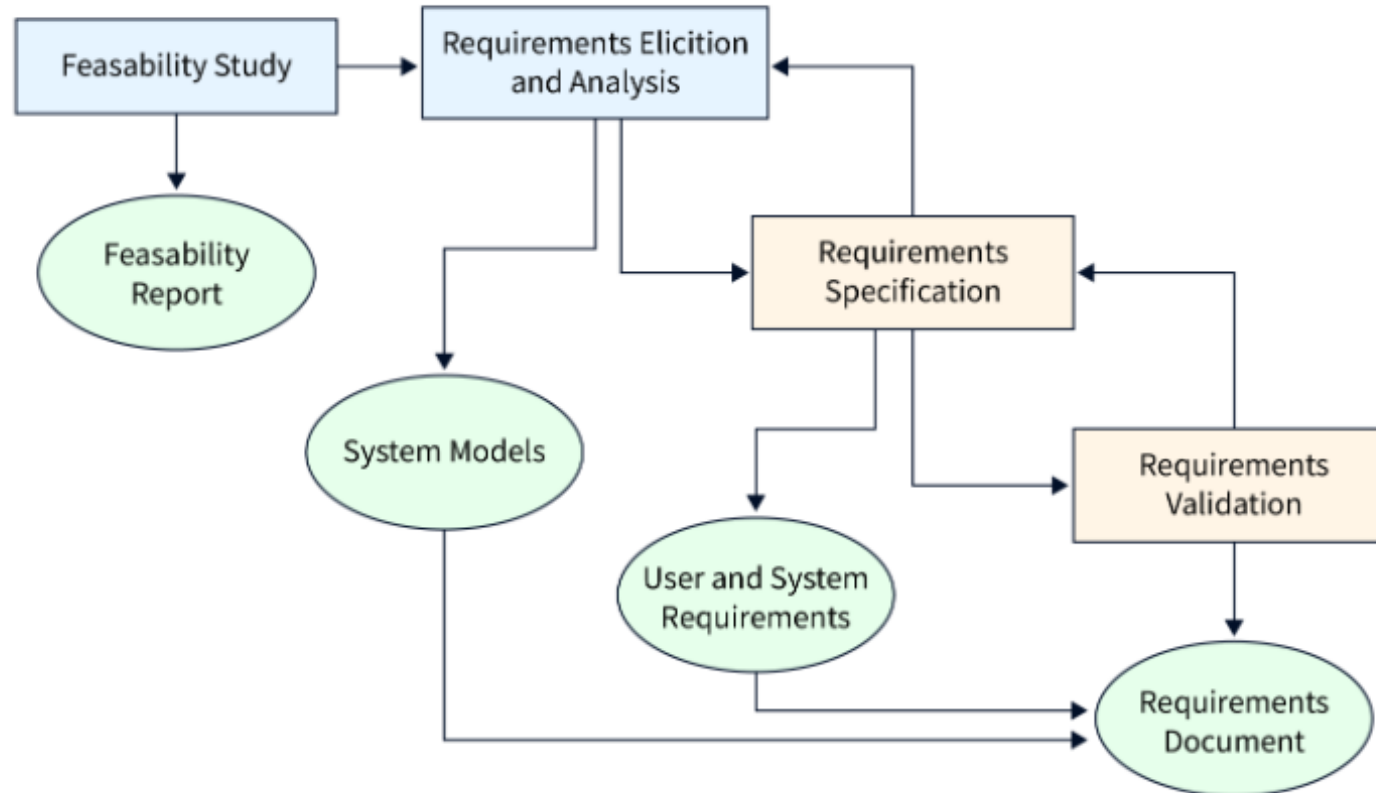
Has an ID, source, owner, and evidence path.

Rule of thumb: if we cannot explain how we will test it, the requirement is not ready.

Requirements engineering

Requirement capture specifies the behavior of the final software system.

Technically,
economically, and
operationally viable



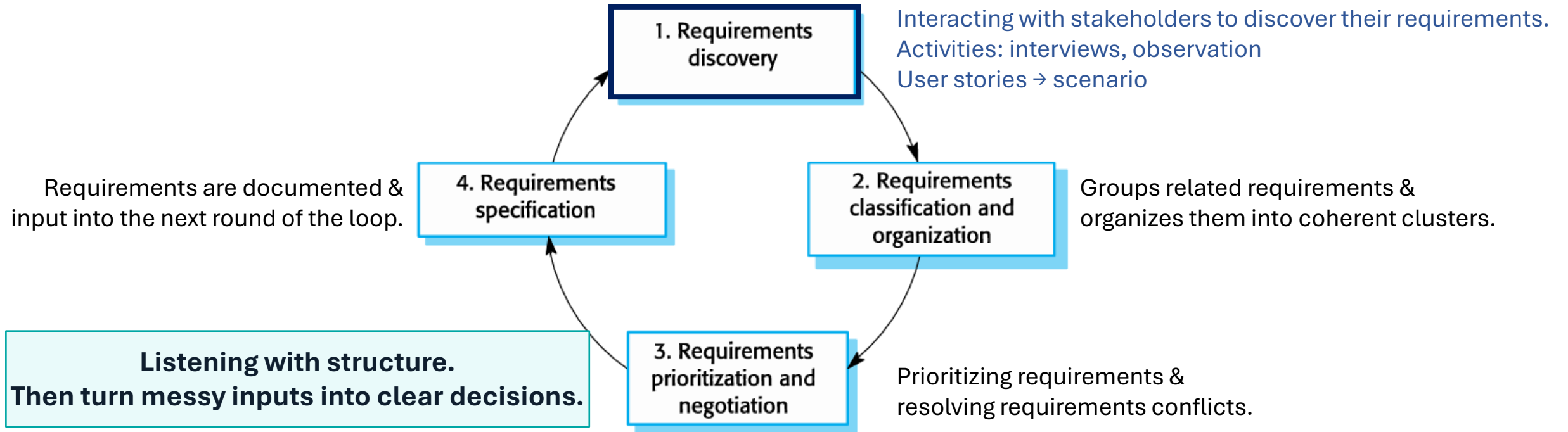
Source: <https://www.scaler.com/topics/requirements-engineering-in-software-engineering/>

**Complete, consistent,
adequate, unambiguous,
verifiable, feasible, necessary**

#1 Requirements elicitation & analysis

Goal: understand the real need before deciding what to build, and make sense of the information and decide what belongs in scope.

- Stakeholder do not know what they really want.
- Collect needs, goals, pain points, constraints, and existing workarounds from stakeholders.
- Conflicted requirements among stakeholders
- Knowledge gaps between stakeholders and developers



Example: how to read a stakeholder request

Turn a paragraph into decisions, not just tasks.

Facts

What the request explicitly says.

Assumptions

What the team believes but has not confirmed.

Questions

What must be asked before scope is safe.

Constraints

Budget, time, platform, data, policy, and environment limits.

Example: “students should browse categories”

Fact: categories are visible.

Assumption: categories come from a database.

Question: who manages categories?

Constraint: Lab 1 exposes only GET /api/categories.

Good requirements work begins by separating *what we know* from *what we still need to learn*.

Example: clarify assumptions before they become defects

Assumptions are not wrong; unreviewed assumptions are risky.

Clarifying questions

Who creates categories?
What data should appear first?
Should categories be sorted?
What happens when the backend is offline?
What counts as a useful error message?

Assumption log

A1: Category data is stored in a database.
A2: Lab 1 uses seed data only.
A3: Browser users are students.
A4: The reviewer can verify behavior with tests and screenshots.

- ✓ Each assumption has an owner or confirmation path.
- ✓ High-risk assumptions become questions or acceptance criteria.

Example: define scope: in, out, and later

Scope prevents the team from building every possible interpretation.

In scope for Lab 1

- Health endpoint
- Category database model
- Idempotent seed
- GET /api/categories
- Check System button
- Loading/success/failure feedback

Out of scope now

- Ticket purchase
- Login/roles
- Payment
- Admin category editing
- Real deployment
- Advanced search/filter

Later backlog

- Create event pages
- User authentication
- Ticket reservation
- Admin dashboard
- Production deployment

Scoping is not deleting ideas. It is deciding what belongs in this increment.

#2 Requirements specification

Goal: write requirements clearly enough to design, build, test, and review.

Writing down the user and system requirements: clear, testable, feasible, traceable.

User story

As a student, I want to view event categories so that I can start browsing events.

Functional req.

REQ-01: The system shall return the list of categories from the database.

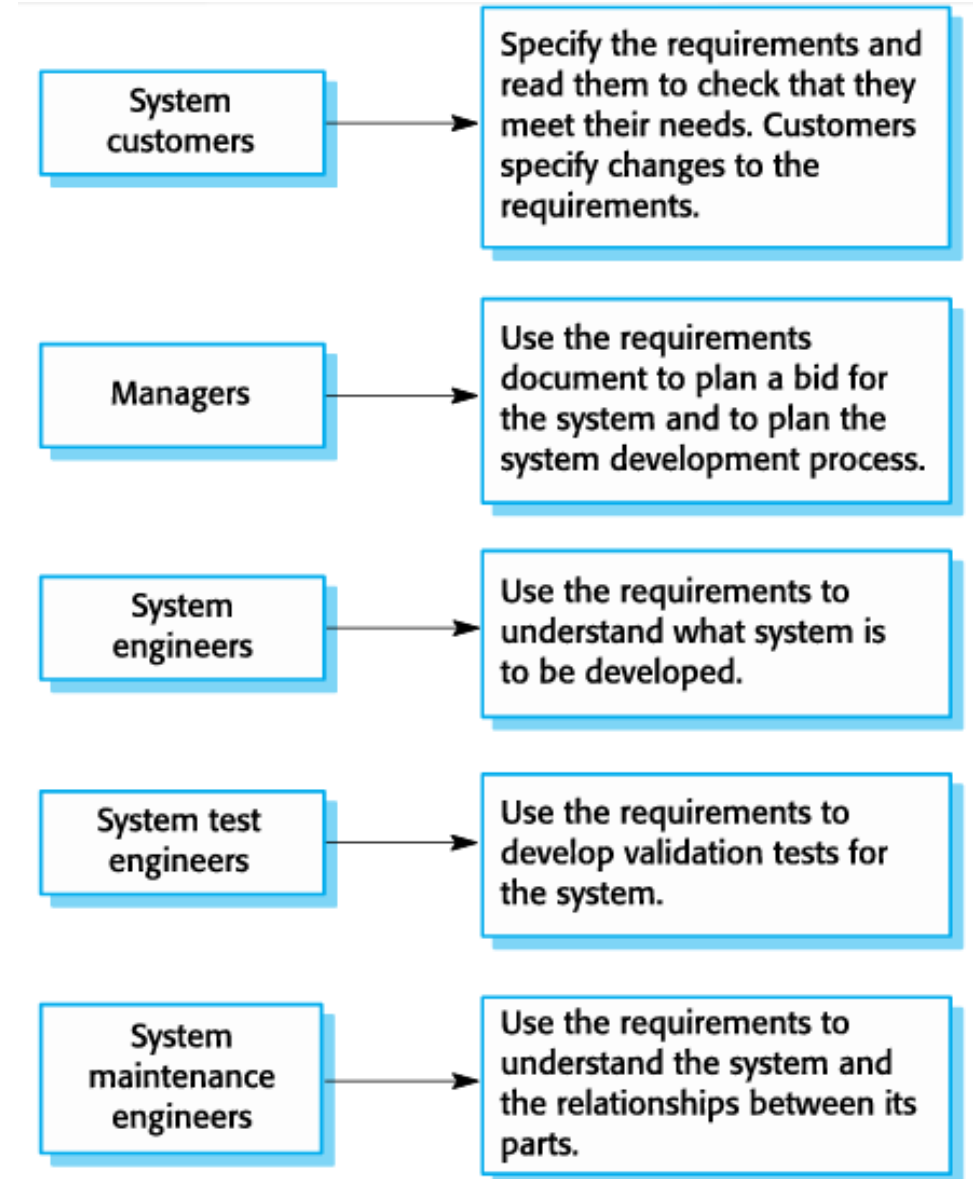
NFR

NFR-01: Category responses should be valid JSON and should not expose database credentials.

Acceptance criteria example:

Given the backend and database are running, when the user clicks [Check System], then the UI displays the API status and category names from /api/categories.

Makes implementable requirements.



Example: acceptance criteria turn requirements into checks

They define what must be true for a feature to be accepted.

Given / When / Then pattern

Given the backend is running and categories exist

When the student clicks [Check System]

Then the UI displays backend status and category names from the database

Good criteria are

observable

specific

testable

independent from implementation details

Poor criteria

The app works.

It looks nice.

AI says it is complete.

Should be reliable.

Acceptance criteria answer: “How will we know this requirement is satisfied?”



Software Requirements Specification (SRS)

Table of Contents

Table of Contents	ii
Revision History	ii
1. Introduction.....	1
1.1 Purpose	1
1.2 Document Conventions	1
1.3 Intended Audience and Reading Suggestions	1
1.4 Product Scope.....	1
1.5 References	1
2. Overall Description.....	2
2.1 Product Perspective	2
2.2 Product Functions.....	2
2.3 User Classes and Characteristics.....	2
2.4 Operating Environment	2
2.5 Design and Implementation Constraints	2
2.6 User Documentation.....	2
2.7 Assumptions and Dependencies.....	3
3. External Interface Requirements	3
3.1 User Interfaces	3
3.2 Hardware Interfaces	3
3.3 Software Interfaces.....	3
3.4 Communications Interfaces.....	3
4. System Features	4
4.1 System Feature 1	4
4.2 System Feature 2 (and so on).....	4
5. Other Nonfunctional Requirements.....	4
5.1 Performance Requirements	4
5.2 Safety Requirements	5
5.3 Security Requirements	5
5.4 Software Quality Attributes	5
5.5 Business Rules	5
6. Other Requirements	5
Appendix A: Glossary	5
Appendix B: Analysis Models.....	5
Appendix C: To Be Determined List.....	6

- A complete document of the requirements of a system/software application
- An agreement between stakeholders

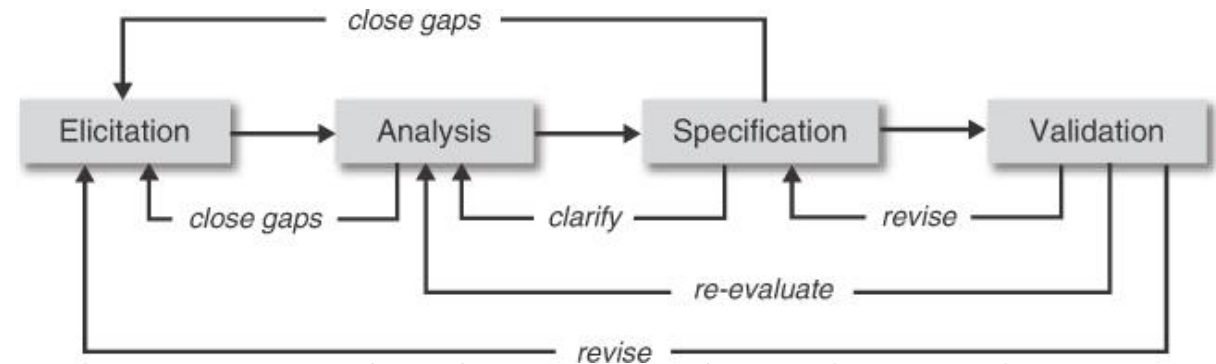
Source: <https://ieeexplore.ieee.org/document/278253>

#3 Requirements validation

Goal: check that requirements are correct before and during implementation.

- Demonstrating that the requirements define the system that the **customer really wants**.
 - Requirements checking
 - **Validity** - Does the system provide the functions which best support the customer's needs?
 - **Consistency** - Are there any requirements conflicts?
 - **Completeness** - Are all functions required by the customer included?
 - **Realism** - Can the requirements be implemented given available budget and technology ?
 - **Verifiability** - Is the requirement realistically testable?
 - **Comprehensibility** - Is the requirement properly understood?
 - **Traceability** - Is the origin of the requirement clearly stated?
 - **Adaptability** - Can the requirement be changed without a large impact on other requirements?
 - Techniques
 - **Review** - stakeholders read the requirement and confirm meaning.
 - **Prototype** - show the expected interaction early, even with simple mock data.
 - **Test-case generation** - check testability.
-
- ```
graph LR; Elicitation --> Analysis; Analysis --> Specification; Specification -- "close gaps" --> Elicitation;
```

## Validation reduces rework.

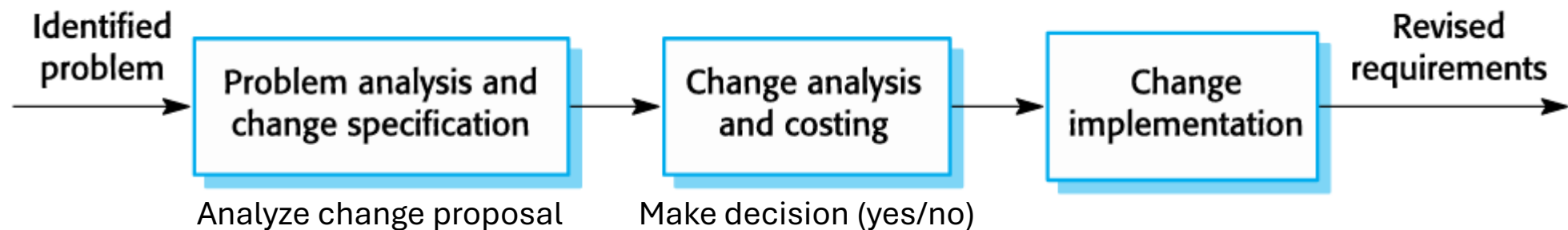


Source: <https://www.informit.com/articles/article.aspx?p=3172443&seqNum=2>

# #4 Requirements change management

Goal: keep requirements visible, traceable, and controlled as the project changes.

- Requirements change - users learn, risks appear, technology constraints emerge, and deadlines become real.
- Management prevents hidden scope creep.
- **Change control** - record the change, reason, owner, impact, and decision to assess the impact of changes.
- **Versioning** - keep **requirement IDs** stable; update status and acceptance criteria.
- A **change management process** - set of activities that assess the impact and cost of changes.
- **Traceability policies** - define the relationships between each requirement, between the requirements and the system design.



# Lightweight Proposal Thinking

Use proposal logic to explain why the feature should exist, not only how to code it.

**Practice - For this course, each feature can have a small proposal: one paragraph plus criteria, risks, and evidence.**

## Problem

What pain or opportunity exists?

## Users

Who experiences the problem and who decides?

## Solution

What product behavior will address it?

## Expected Value

What improves if the solution works?

## Scope

What is included now, later, or not?

## Risks

What might fail or be uncertain?

## Effort

How hard is it roughly?

An early estimate used to choose scope and manage risk.  
*Examples: API + DB, UI states, test + evidence, integration*

## Evidence

What will prove completion?

# Feature Scoping

Choose the smallest useful increment that can be implemented, reviewed, and tested.

## MVP (Minimum Viable Product) mindset

**Minimum** – only essential

**Viable** – usable

**Product** – live software for users

*Small but coherent, not incomplete fragments.*

## Priority logic

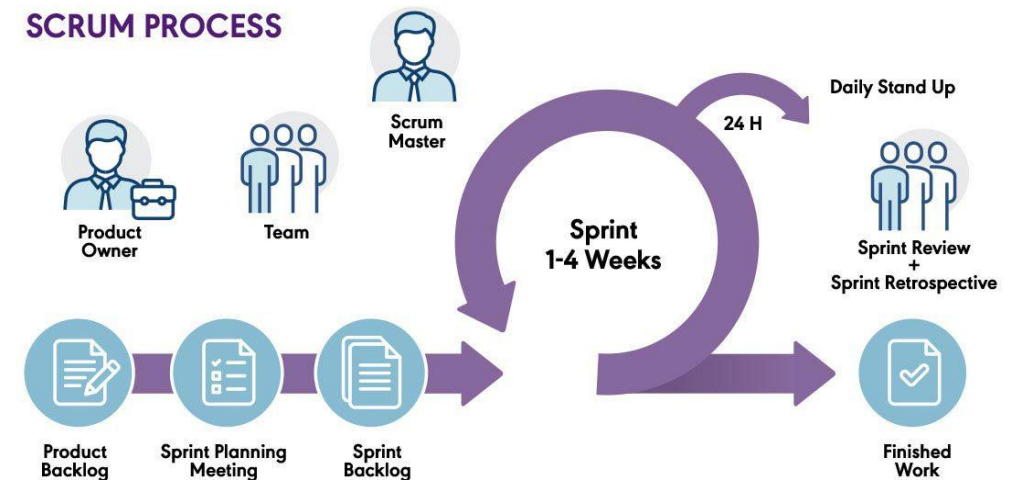
Value, risk, effort, dependency, and learning.

*Do the work that teaches the most or reduces the most uncertainty early.*

## ? Lab 1 ordering logic

Good scoping makes progress visible.

## SCRUM PROCESS



# Traceability concept

Every change should have a reason and evidence.

- Traceability **connects decisions to evidence**.
- The goal is not paperwork; the goal is reviewable engineering decisions.

## Traceability answers

Why was this code written?

Which requirement does it satisfy?

How was it verified?

## Common failure

Code exists, but no one can connect it to the original request, acceptance criteria, test output, or demo screenshot.

## Reason

Where did this requirement come from?

## Implementation

Which design, task, and files realize it?

## Evidence

Which tests, screenshots, and records prove it?

## Requirement

what must be true

## Design

▶ how it will work

## Task

▶ Issue / branch

## Code

▶ changed files

## Test

▶ evidence

## Release

▶ accepted change

## Demo

▶ visible proof

# Example: a lightweight traceability matrix is enough for Lab 1

Use IDs and evidence links; do not create heavy documentation.

| problem, scope,<br>acceptance criteria |         |          |           | commands and<br>passing output |             |                    |
|----------------------------------------|---------|----------|-----------|--------------------------------|-------------|--------------------|
| Req ID                                 | Source  | Issue    | Branch    | Code                           | Test        | Demo evidence      |
| FR-HEALTH                              | request | Issue #2 | feature/2 | health route                   | health.test | API screenshot     |
| FR-CAT                                 | request | Issue #3 | feature/3 | Category model                 | seed/test   | DB + API evidence  |
| FR-UI                                  | request | Issue #4 | feature/4 | App component                  | App.test    | browser screenshot |

- The matrix is useful only when it points to real artifacts: Issues, PRs, changed files, test output, screenshots, and reviewer comments.
- Pull Request: review comments and linked Issue
- Evidence chain: what reviewers should be able to inspect.
- A claim of **done** must be backed by visible artifacts.

**Done = accepted by review + satisfies criteria + tests pass + evidence documented.**

# Why requirements engineering matters more with AI agents

Code is easier to generate; the wrong product is still expensive.

## Without requirements

- Fast coding
- Unclear target
- Generated complexity
- Stakeholders reject the outcome

## With requirements

- Shared product intent
- Smaller increments
- Clear acceptance criteria
- Better agent prompts
- Traceable evidence

**Requirement = a documented need, capability, constraint, or quality condition the software must satisfy.**



# Better specifications create better AI prompts

Give the agent boundaries, not only wishes.

## Weak prompt

“Build the category feature.”

## Stronger prompt

“Implement FR-CAT-01. Add GET /api/categories returning DB categories as JSON. Use Prisma Client. Add Supertest coverage. Do not hard-code categories in React.”

## Ask for a plan first

List files to change, tests to add, assumptions, and commands to run before code changes.

## Review the evidence

Inspect the diff, run tests yourself, check the UI manually, and document remaining risks.

**Let's prepare for our lab session.**

# Project structure and toolchain

Keep frontend, backend, docs, tests, and environment files clearly separated.

```
toktickit/
├── client/
├── server/
│ ├── prisma/
│ ├── src/
│ └── tests/
│ └── lab-01/
├── docs/
│ └── lab-01/
├── .gitignore
└── README.md
```

## Separation of purpose

client: frontend app

server: backend API, Prisma, tests

docs: screenshots, notes, evidence

## Package management

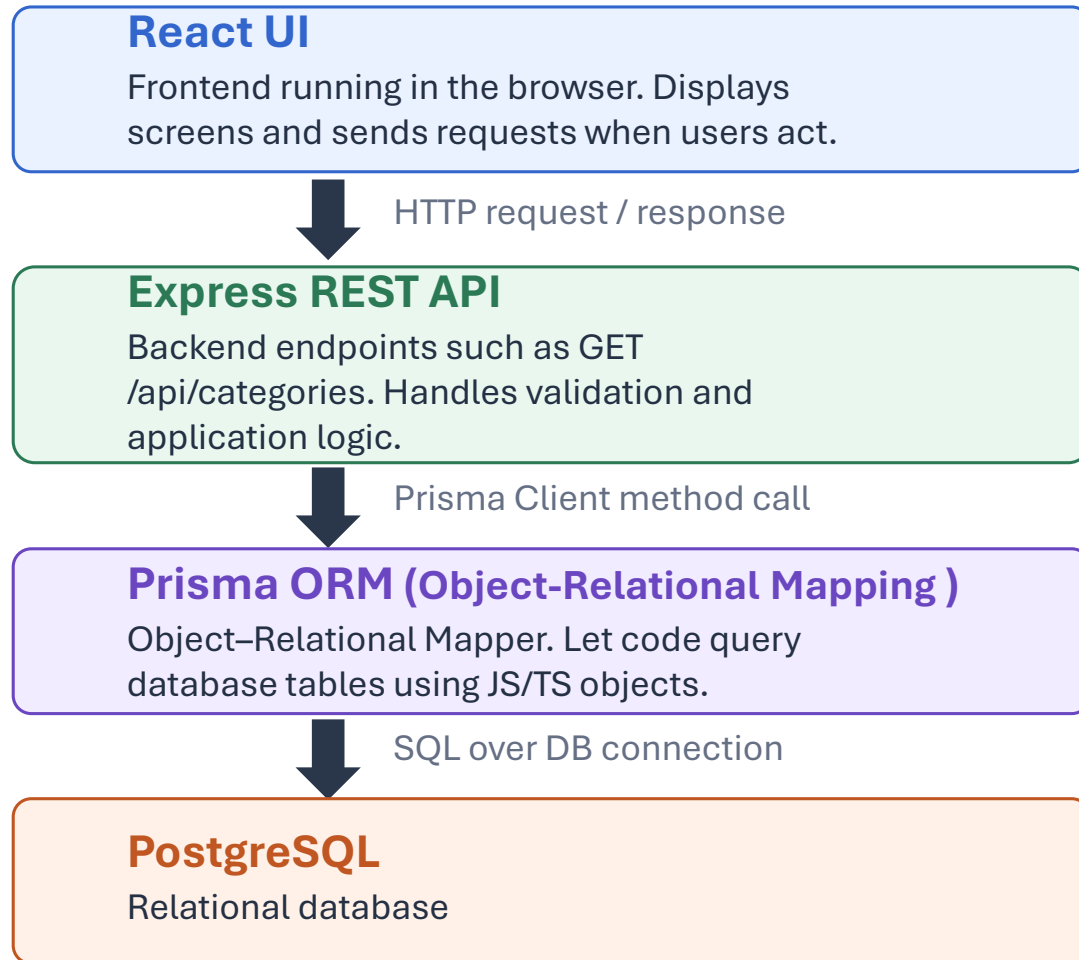
Install dependencies separately in client and server folders.

## Common scripts

places files in dedicated root-level directories

development · build · test · seed · migration

# Full-stack architecture review



## Key rule

React should not talk directly to PostgreSQL. The frontend calls the API.

## ORM idea

The backend uses Prisma to query and update database tables with TypeScript-style code.

# Frontend quick review: React + TypeScript + Vite + Bootstrap

Students only need the basics needed for Lab 1 behavior.

## React

Build **UI** as **components** and re-render when state changes.

## TypeScript

Adds types to reduce mistakes in props, state, and API data.

## Vite

Fast dev **server** and build **tool** for frontend projects.

## Bootstrap

Ready-made **CSS** classes for layout, buttons, alerts, and cards.

## Component structure

Keep App, UI components, and API helpers readable.

## Scripts

npm install · npm run dev · npm run build · npm test

**Frontend responsibility: show title, [Check System] button, loading, success data, and useful failure feedback.**

# Backend quick review: Node.js + Express + TypeScript

## **Node.js** (server runtime env)

Runs JavaScript/TypeScript server code.

## **Express** (web framework)

Defines routes such as GET /api/health and GET /api/categories.

## **TypeScript**

Makes request, response, & data structures easier to reason about.

## **Route organization**

Group endpoint handlers so files stay understandable.

## **Server startup**

Keep createApp() separate from app.listen() for testing.

Creates an isolated application object.  
Binds the application to a specific network port.

## **Middleware + env**

Use JSON middleware and .env for config.

**Backend responsibility: return JSON, status codes, useful errors, and database-backed categories.**

# REST API fundamentals: quick review

An API is the contract between frontend and backend.

Representational State Transfer | Application Programming Interface

## HTTP method (operation)

GET reads data; POST creates data.

## Endpoint

URL path such as /api/categories.

## Resource

Thing exposed by the API, such as categories.

## Route

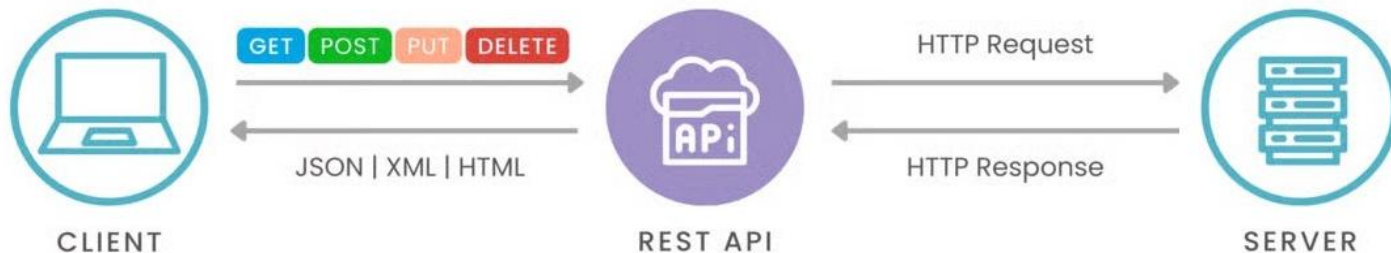
Backend code that handles an endpoint.

## Status code

200 OK, 404 Not Found,  
500 Server Error.

## JSON payload

Structured request or response data.



Source: <https://www.skiplevel.co/blog/part-2-rest-api-components-how-to-read-them>

```
GET /api/health → 200 OK
```

```
{
 "status": "ok",
 "service": "TokTickIT API"
}
```

**Separation principle: UI chooses how to display data; API defines how data is requested and returned.**

More reading: <https://www.skiplevel.co/blog/part-2-rest-api-components-how-to-read-them>

# Express route implementation review

Create the app, create routes, send JSON, and return status codes.

```
// app.ts
import express from "express";

export function createApp() {
 const app = express();
 //JSON middleware
 app.use(express.json());
 //Define API route
 app.get("/api/health", (_req, res) =>
 {
 //HTTP response code = 200 & JSON response
 res.status(200).json({
 status: "ok",
 service: "TokTickIT API"
 });
 });

 return app;
}
```

## Testing-friendly structure

Keep app construction separate from server startup so Supertest can import createApp().

```
// server.ts
const app = createApp();
app.listen(PORT);
```

## Basic error handling

Return useful JSON errors and appropriate status codes. Avoid silent blank screens.



# PostgreSQL review and Prisma fundamentals

The database stores data; Prisma maps TypeScript-style operations to SQL.

**Table** - collection of rows with the same columns

**Row** - one record, such as one category

**Column** - one field, such as name

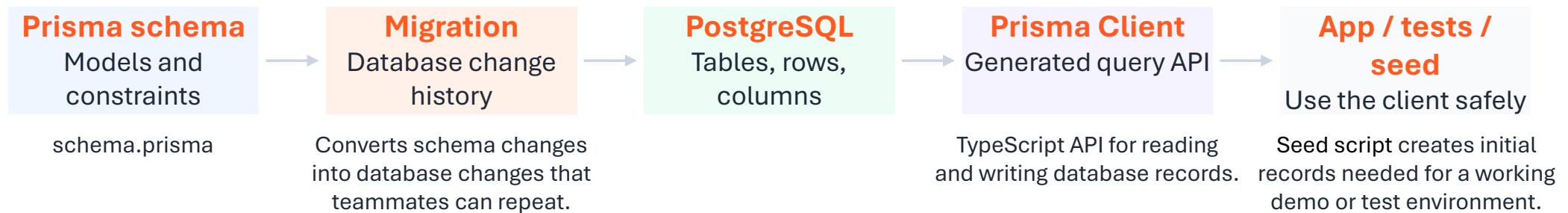
**Primary key** - unique identifier for each row

**Unique constraint** - prevents duplicate category names

**Timestamp** - records when data was created

## Predictable workflow in Lab 1

- **Initialize** - create Prisma config & schema files.
- Connect database in .env and keep it private.
- **Migrate** - apply the Category table to PostgreSQL.
- **Generate** client - refresh TS query API & types.
- Query + **seed** - read categories & seed without duplicates.



```
model Category {
 id Int @id
 @default(autoincrement())
 name String @unique
 createdAt DateTime @default(now())
}
```

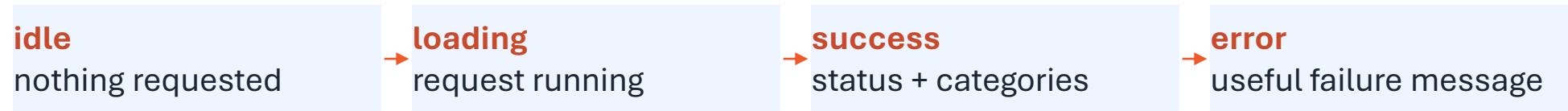
```
// Query categories
const categories = await
 prisma.category.findMany({
 orderBy: { name: "asc" }
 });
```

```
// Idempotent seed: safe to run
// more than once
await prisma.category.upsert({
 where: { name },
 update: {},
 create: { name }
});
```

Use the existing row instead of create duplicated data.

# React data fetching and basic UI behavior

Students should recognize loading, success, and failure as separate states.



```
async function checkSystem() {
 setLoading(true);
 setError(null);
 try {
 const health = await fetch("/api/health");
 const categories = await fetch("/api/categories");
 // render returned JSON data
 } catch {
 setError("Cannot reach the backend.");
 } finally {
 setLoading(false);
 }
}
```

- ✓ Show [Check System].
- ✓ Display backend status.
- ✓ Display categories from database.
- ✓ Show loading feedback.
- ✓ Show useful failure feedback.

**Frontend data must come from the API, not from hard-coded arrays in React.**

# Automated testing fundamentals

Tests should check behavior that matters to users and reviewers.

## Vitest

**Frontend tests:** render a component, query elements, simulate button click, mock API response, check loading/success/failure.

## Supertest

**Backend tests:** check whether the webserver sending/receiving data correctly. *Call Express routes without a browser, assert status code, content type, and JSON body.*

## Test quality

- Arrange · Act · Assert
- One behavior per test
- Test behavior, not internals implementation

```
// Vitest idea
render(<App />);
await user.click(screen.getByText("Check System"));
expect(await screen.findByText("Workshop"))
 .toBeInTheDocument();
```

- Mounts the main `<App />` component.
- Simulates a user clicking on an element that contains the exact text “Check System”.
- Waits for the text “Workshop” to appear on the screen, then asserts that it exists.

```
// Supertest idea
await request(app)
 .get("/api/health")
 .expect(200)
 .expect({ status: "ok", service: "TokTickIT API" });
```

- Starts a test agent using your local backend app.
- Sends a GET HTTP request.
- Asserts that the HTTP response status code = 200.
- Checks the incoming JSON response body against the provided object.